Project 3 – Informe de correcciones y cambios

Hecho por:

Juan Diego Montañez (202420709) y Natalia Salazar Tena (202421753)

Correcciones del proyecto 2:

Ajustamos la gramática para que los saltos de línea puedan separar instrucciones dentro de un bloque. Así los ejemplos de Objectilang pueden ejecutarse tal como están escritos, sin añadirles puntos y coma. El ";" sigue siendo válido como separador. También actualizamos el conversor para que recorra las instrucciones del bloque y las conserve en el AST en el mismo orden.

Sobre la conversión del CST al AST, completamos las conversiones señaladas usando "unparse" para extraer el texto de los nodos del CST. Por ejemplo, al procesar val edad= 25, el conversor obtiene edad del nodo del identificador y construye integerLiteral(25) a partir del texto del literal. Para los enteros con toInt(texto,10). Verificamos que 08 y 010 producen intergerLiteral(8) e intergerLiteral(10).

Cambios realizados para la entrega de proyecto 3:

Reutilizamos la gramática y el AST de Objectilang para ejecutar programas completos. El módulo Main recibe un archivo .ol, lo analiza y convierte de un CST a un AST. Después de verificar los tipos, el generador traduce el programa a código Rascal, que se ejecuta y muestra el resultado en consola.

Ampliamos la gramática para admitir anotaciones de tipo como val edad: Int = 25, con soporte para Int, Bool, Char y String. También incorporamos objetos con parámetros genéricos como Contenedor [T]. El verificador comprueba que los valores, los argumentos y los retornos sean compatibles con los tipos declarados. Por ejemplo, al crear un Contenedor[Int], el verificador interpreta T como Int en sus métodos y rechaza un argumento tipo String donde se espera un entero.

Añadimos una regla que comprueba que los elementos referenciados estén definidos. Por ejemplo, si un objeto declara "extends NoExiste" y ese objeto padre no está definido, el verificador reporta el error antes de ejecutar el programa. También se comprueba que existan los tipos, campos y métodos utilizados.

Instalamos TypePal como dependencia de Maven y comprobamos que Rascal pudiera importar la biblioteca. Está incluido en el archivo Pom.xml. Las reglas de verificación de Objectilang se implementaron en el módulo TypeChecker.rsc. Probamos los 30 programas de ejemplo, de los cuales 27 válidos se ejecutaron y los 3 inválidos se rechazaron por errores previstos. Se rechazaron por una declaración incompleta, una asignación con tipos incompatibles y una referencia a un objeto padre inexistente. Son los siguientes archivos: prueba-parser-invalido.ol, punto5-error-tipos.ol, punto6-error-referencia.ol .